Move Semantics

Value categories, what `std::move` really does, writing move operations, copy elision, and the traps. Assumes OOP (the Rule of Five).

Lvalues, rvalues and value categories

An **lvalue** has a name and an address (`x`, `v[0]`). An **rvalue** is an unnamed temporary (`5`, `a + b`, a returned value): about to vanish, so its contents can be stolen safely.

CATEGORY	ROUGHLY	EXAMPLES
lvalue	a named object	`x`, `v[0]`
prvalue	an unnamed temporary	`5`, `a + b`, a call that returns by value
xvalue	an object marked as movable	`std::move(x)`

"Rvalue" means prvalue or xvalue. Which reference a value can bind to:

REFERENCE TYPE	BINDS TO	TYPICAL USE
`T&`	lvalues only	modify the caller's object
`const T&`	lvalues **and** rvalues	read-only, avoid a copy
`T&&` (rvalue reference)	rvalues only	move constructors and move assignment

```cpp
void f(const std::string& s);   // accepts anything, copies nothing
void g(std::string&& s);        // accepts only temporaries: "I may take your contents"

std::string name = "abc";
f(name);              // OK
// g(name);           // ERROR: name is an lvalue
g(std::string("x"));  // OK: temporary
g(std::move(name));   // OK: std::move says "treat name as a temporary"

int x = 0;
static_assert(std::is_same_v<decltype((x)),            int&>);    // double parentheses report the category:
static_assert(std::is_same_v<decltype((std::move(x))), int&&>);   // lvalue int&, xvalue int&&, prvalue plain int
static_assert(std::is_same_v<decltype((x + 1)),        int>);
```

A NAME IS ALWAYS AN LVALUE Inside `g`, `s` is an lvalue despite its `&&` type. Pass it on with `std::move(s)` (or `std::forward`, see Templates).

READ MORE

C++ Primer (Lippman, Lajoie, Moo)

- 4.1 Fundamentals (PDF pages 163 to 167)
- 13.6 Moving objects (PDF pages 560 to 579)

C++ Move Semantics (Josuttis)

- 8.1.2 to 8.1.3 and 8.4 to 8.5 Value categories since C++11 and C++17; when lvalues become rvalues and the reverse (PDF pages 145 to 148 and 153 to 154)
- 8.2 to 8.3 and 8.6 Special value-category rules; binding references; checking with decltype (PDF pages 148 to 153 and 154 to 156)

Copy versus move

Copy: b = a



allocates and copies a million elements

Move: b = std::move(a)



b stole the pointer: two assignments, no copying.

The moved-from object is valid but unspecified: destroy it or assign to it.

std::move only casts; the move constructor does the stealing.

Copying a million-element vector copies a million elements. If the source is a dying temporary, that is wasted work. A **move** just *steals the internal pointer*.

STEP THROUGH **Copy vs move: what std::move really does**

```
1  std::vector<int> a(1000, 7);
2  std::vector<int> b = a;
3  std::vector<int> c = std::move(a);
```

STEP	RUNS	WHAT HAPPENS	MEMORY AFTER (CHANGED IN BOLD)
1	line 1 `std::vector<int> a(1000, 7);`	**a** is a small object (pointer + size + capacity) that owns a big heap buffer. It is an **lvalue**: it has a name and an address.	**`a.ptr = 0x5000`** | **`a.size = 1000`** | **`buffer A = 7, 7, 7 ... (heap)`**
2	line 2 `std::vector<int> b = a;`	In `b = a` the argument `a` is an **lvalue**, so it binds to `const vector&` and the compiler picks the **copy constructor**. Overload resolution only looks at the *kind* of expression you pass.	`a.ptr = 0x5000` | `a.size = 1000` | `buffer A = 7, 7, 7 ... (heap)`
3	line 2 `std::vector<int> b = a;`	**Copy:** a new buffer is allocated and all 1000 ints are copied. Two independent owners. Correct, but O(n).	`a.ptr = 0x5000` | `a.size = 1000` | **`b.ptr = 0x6000`** | **`b.size = 1000`** | `buffer A = 7, 7, 7 ... (heap)` | **`buffer B = 7, 7, 7 ... (heap)`**
4	line 3 `std::vector<int> c = std::move(a);`	**std::move(a) runs first, and it moves nothing.** It is just `static_cast<vector<int>&&>(a)`: it hands back an **rvalue reference to the very same object** (follow the arrow: same address, no copy, no data touched). The *expression* changed from "lvalue a" to "xvalue, an rvalue you may steal from".	`a.ptr = 0x5000` | `a.size = 1000` | `b.ptr = 0x6000` | `b.size = 1000` | **`std::move(a) = refers to a`** | `buffer A = 7, 7, 7 ... (heap)` | `buffer B = 7, 7, 7 ... (heap)`
5	line 3 `std::vector<int> c = std::move(a);`	Now overload resolution sees an **rvalue**. `vector(vector&&)` (the **move constructor**) is a better match than `vector(const vector&)`, so it is chosen. That constructor, not `std::move`, does the stealing.	`a.ptr = 0x5000` | `a.size = 1000` | `b.ptr = 0x6000` | `b.size = 1000` | `std::move(a) = refers to a` | `buffer A = 7, 7, 7 ... (heap)` | `buffer B = 7, 7, 7 ... (heap)`
6	line 3 `std::vector<int> c = std::move(a);`	**Move:** no allocation, no element copying. **c** takes over buffer A's pointer and size, and **a** is left empty (valid but unspecified: assign to it or let it die, do not read it). O(1). Had `a` been `const`, the cast would give `const vector&&`, which cannot bind to the move constructor, so it would **copy** silently.	**`a.ptr = nullptr`** | **`a.size = 0`** ~~**`std::move(a) = gone (temporary)`**~~ | **`c.ptr = 0x5000`** | **`c.size = 1000`** | `b.ptr = 0x6000` | `b.size = 1000` | `buffer A = 7, 7, 7 ... (heap)` | `buffer B = 7, 7, 7 ... (heap)`

- `std::move(x)` **moves nothing**: it is a cast to an rvalue reference meaning "you may steal". The move constructor or assignment does the stealing.
- Overload resolution only looks at the *kind* of expression: an lvalue picks the copy, an rvalue picks the move.
- A moved-from object is **valid but unspecified**: destroy it or assign a new value, do not read it. Compiled: a moved-from `std::string` had size 0, and assigning to it worked.

KEY POINT "What does `std::move` do?" Nothing at runtime: an unconditional cast to an rvalue reference. The move constructor does the work.

READ MORE

C++ Primer (Lippman, Lajoie, Moo)

- 13.6 Moving objects (PDF pages 560 to 579)

Effective Modern C++ (Meyers)

- Item 23, Understand std::move and std::forward (PDF pages 176 to 182)

C++ Move Semantics (Josuttis)

- 1.1 Motivation for move semantics (PDF pages 21 to 36)
- 1.2 to 1.4 Implementing move semantics; copying as a fallback; const objects (PDF pages 36 to 41)
- Chapter 2 Core features of move semantics (PDF pages 43 to 52)
- 6.1 to 6.2 Moved-from states (required and guaranteed states; destructible and assignable) (PDF pages 107 to 115)

Writing move operations

A class that owns a raw resource needs the Rule of Five (OOP lesson). The move operations steal and leave the source safely empty:

```
class Buffer {
    int* data_; std::size_t n_;
public:
    Buffer(Buffer&& o) noexcept                                  // steal
and reset, in one line each
        : data_(std::exchange(o.data_, nullptr)), n_(std::exchange(o.n_, 0))
{}

    Buffer& operator=(Buffer o) noexcept {                       // copy-
and-swap: by value covers copy AND move
        std::swap(data_, o.data_); std::swap(n_, o.n_);
        return *this;
    }
    /* ... constructor, destructor, copy constructor ... */
};
```

- `std::exchange(x, v)` returns the old `x` and sets it to `v`: the idiom for "steal and reset".
- **Mark moves** `noexcept`. `std::vector` moves its elements on reallocation only if the move cannot throw; otherwise it copies them (exception safety).
- The by-value `operator=` handles self-assignment and gives the strong guarantee. The classic pair `operator=(const T&)` and `operator=(T&&)` is faster when you can reuse the existing storage.
- If you can avoid owning a raw resource, do (Rule of Zero): `vector` and `unique_ptr` members make the compiler-generated moves correct.

READ MORE

C++ Primer (Lippman, Lajoie, Moo)

- 13.6 Moving objects (PDF pages 560 to 579)

Programming: Principles and Practice (Stroustrup)

- 17.4 to 17.5 Copying and moving; essential operations (PDF pages 501 to 512)
- 17.6 to 17.9 Other operations, remaining Vector problems, changing size (PDF pages 513 to 523)

Effective Modern C++ (Meyers)

- Item 14, Declare functions noexcept if they won't emit exceptions (PDF pages 108 to 115)

C++ Move Semantics (Josuttis)

- 6.3 Dealing with broken invariants (PDF pages 115 to 124)

Returning objects: elision and `std::move`

Compiled with a counting type (copies / moves performed):

WHAT YOU WRITE	COPIES	MOVES	WHY
`return Counter();`	0	0	guaranteed elision (C++17): built in the caller's storage
`Counter c; return c;`	0	0	NRVO: allowed, not required, but done in practice
`return std::move(c);`	0	**1**	blocks NRVO: an extra move, so never write it
return a by-value **parameter**	0	1	no elision for parameters, but it is moved, not copied
`return member;`	**1**	0	a member is never moved implicitly

`return std::move(member);`	0	1	here `std::move` is right: the object keeps living, you take its member

```
Buffer make()          { Buffer b(1000); return b; }              // fine: elided or moved automatically
Buffer take(Source& s) { return std::move(s.buffer); }            // member of an object that outlives the call: move it out
```

> RULE Return locals and temporaries plainly. Use `std::move` on a **member**, or on something that is not the local you are returning by name.

> READ MORE
>
> **Effective Modern C++ (Meyers)**
>
> - Item 25, Use std::move on rvalue references, std::forward on universal references (PDF pages 186 to 195)

Sink parameters and move traps

Taking a parameter you will store

```
class Person {
    std::string name_;
public:
    explicit Person(std::string n) : name_(std::move(n)) {}    // by value, then move into the member
};
```

Compiled with a counting type: `Holder(lvalue)` and `Holder(rvalue)` together made **1 copy and 2 moves**: the lvalue is copied once into the parameter, and each call moves once into the member. For a **constructor that stores its argument**, take it by value and move. Not for setters: the extra construction can cost more than assigning.

Traps

TRAP	WHAT HAPPENS
`std::move` on a `const` object	**copies**: a `const T&&` cannot bind to `T&&`, so the copy constructor is chosen (compiled: 1 copy, 0 moves)
a type with no move operations	silently copies
using an object after `std::move`	valid but unspecified; only destroy or assign to it
`std::move` on a forwarding reference parameter	moves an lvalue the caller still wanted: use `std::forward`
moving inside a loop	the second iteration sees a moved-from object

`push_back(lvalue)` copies, `push_back(temporary)` moves, `emplace_back(args ...)` builds in place (compiled together: 1 copy, 1 move).

> **MOVE ONLY THE LAST USE** After `auto b = std::move(a);` treat `a` as empty. Never return `const` values by value: it disables the move.

READ MORE

Effective Modern C++ (Meyers)

- Item 29, Assume move operations are not present, not cheap, and not used (PDF pages 221 to 225)
- Item 41, Consider pass by value for copyable, cheap-to-move parameters that are always copied (PDF pages 299 to 310)

C++ Move Semantics (Josuttis)

- 4.1 to 4.2 Avoid objects with names; avoid unnecessary std::move (PDF pages 77 to 79)
- 4.3.1 to 4.3.4 Initializing members with move semantics (PDF pages 79 to 90)
- 4.3.5 to 4.3.6 Member initialization summary; always pass by value and move? (PDF pages 90 to 93)

Cheat sheet

Answer aloud, then open.

▼ **What does `std::move` actually do?**

Nothing at runtime: it is an unconditional cast to an rvalue reference. It enables the move constructor or move assignment to be chosen. The moved-from object stays valid but unspecified. On a `const` object it silently copies, because a `const` rvalue cannot bind to a move constructor.

▼ **What are lvalues, prvalues and xvalues?**

An lvalue is a named object with an address. A prvalue is an unnamed temporary (a literal, `a + b`, a call returning by value). An xvalue is an object marked as movable, such as `std::move(x)`. Rvalue means prvalue or xvalue. A named rvalue-reference parameter is still an lvalue inside the function.

▼ What is copy elision / RVO?

The compiler constructs a returned object directly in the caller's storage, skipping the copy or move. In C++17 it is guaranteed when returning a temporary (prvalue). NRVO for a named local is allowed but not required. Do not write `return std::move(local)`: it prevents it and costs a move.

▼ Why mark a move constructor `noexcept`?

`std::vector` moves its elements on reallocation only if the move cannot throw; otherwise it copies them to keep the strong guarantee. Without `noexcept` every growth becomes a round of copies.

▼ What state is a moved-from object in?

Valid but unspecified: you may destroy it or assign a new value to it, but not rely on its contents. For standard types it is usually empty.

▼ How do you write a class that stores a string argument efficiently?

Take the parameter by value and `std::move` it into the member. An lvalue costs one copy plus one move, an rvalue costs one move, and there is a single function to write.

▼ What is the difference between `push_back` and `emplace_back`?

`push_back` takes an existing object (copied or moved into the container). `emplace_back` forwards its arguments and constructs the element in place, saving a temporary. Both reallocate the same way.